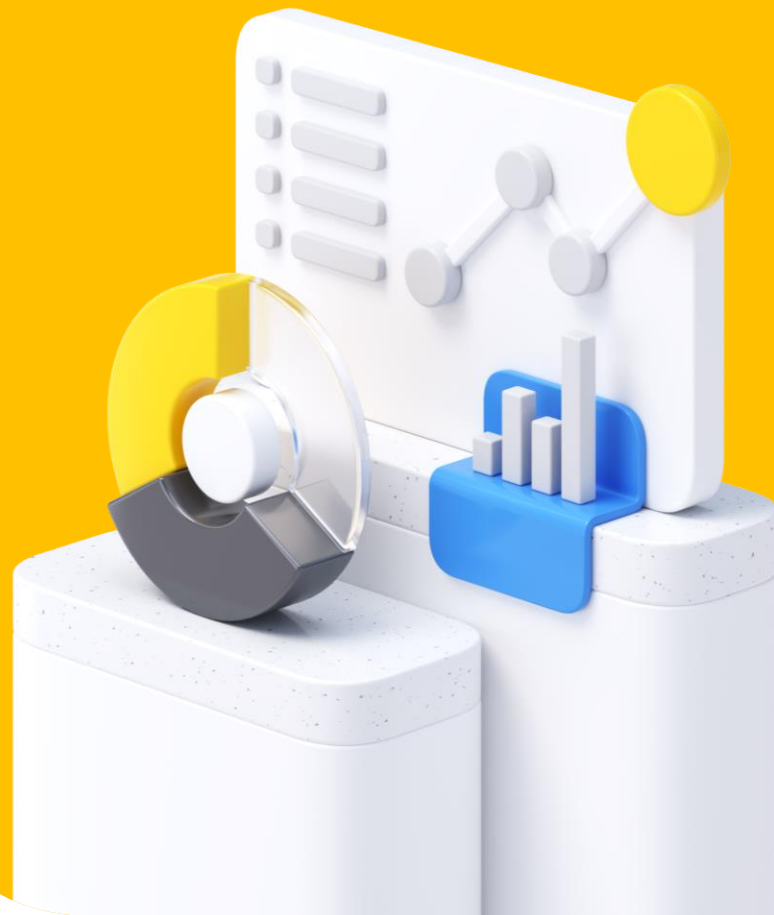




Лекция 5

Память и диспетчеризация



Обо мне



- Катаюсь на велосипеде и люблю плавать
- Занимаюсь образовательными проектами
- Почти всегда можно найти в офисе
- Переехал из Питера, 52!

Опыт

06.2018
06.2020

СПБГМУ (бакалавриат)
СПБГМУ (магистратура)

10.2020

Syncleo (iOS developer)

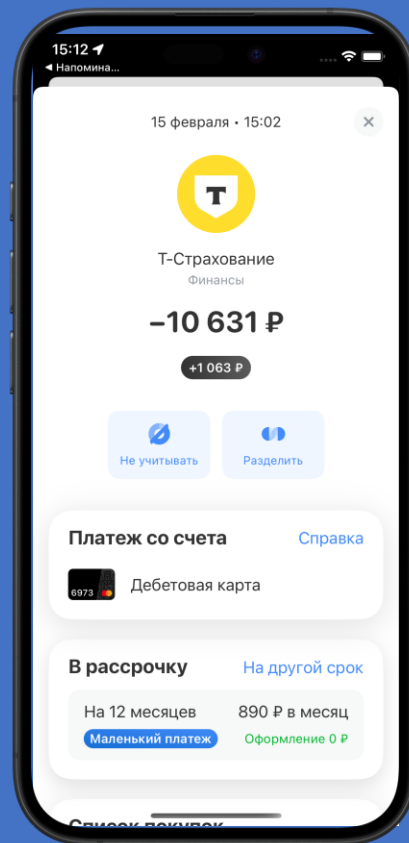
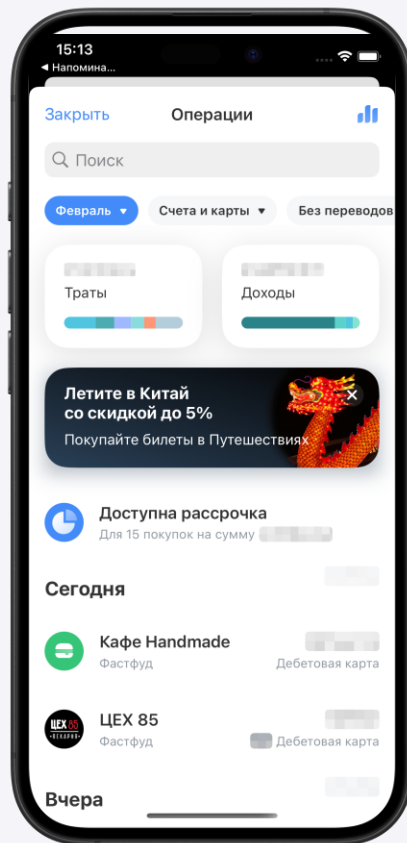
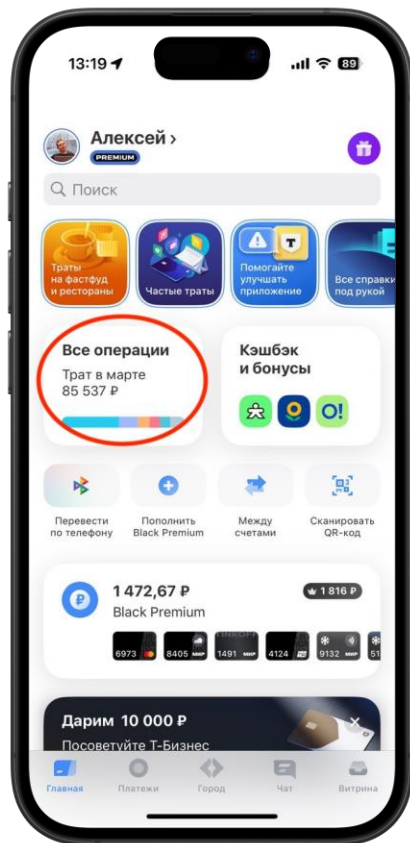
10.2021

Tinkoff Fintech школа

03.2022

Tinkoff iOS developer (Operations)

Наши экраны



План урока

Память

Демо

Диспетчеризация

Память



Поднимите руки кто помнит!



Что ты ел сегодня на завтрак?

Поднимите руки кто помнит!



Что ты ел вчера на завтрак?

Поднимите руки кто помнит!



Что ты ел неделю назад на завтрак?

Поднимите руки кто помнит!



Что ты ел месяц назад на завтрак?

Garbage collector



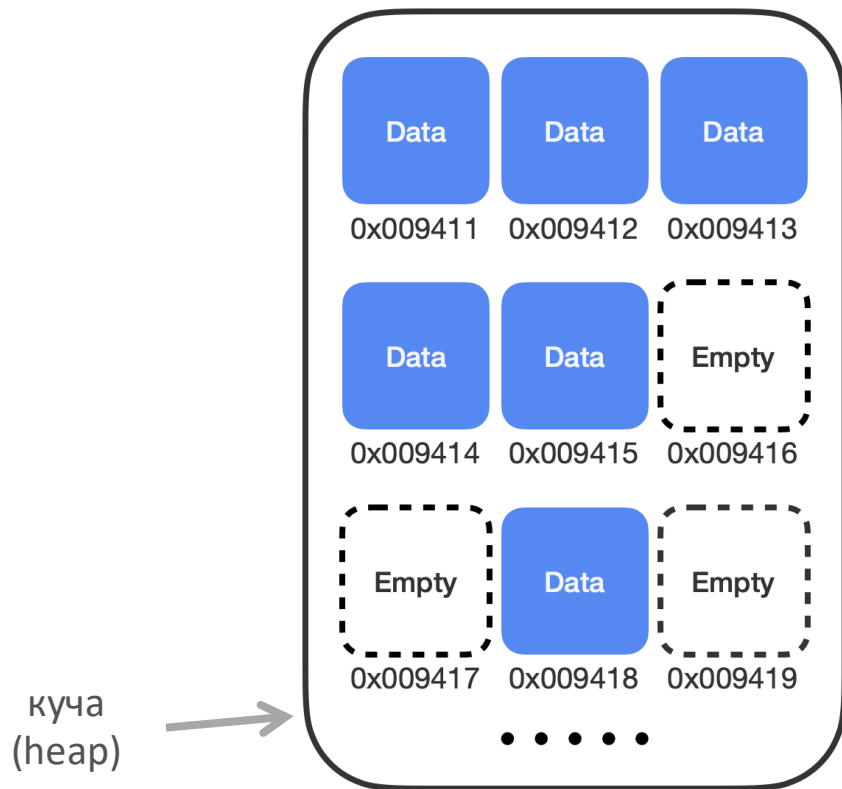
Value & Reference types

Value Types	Reference Types
Struct Enum Int Double String Set Tuple Array Dictionary	Class Function Closure

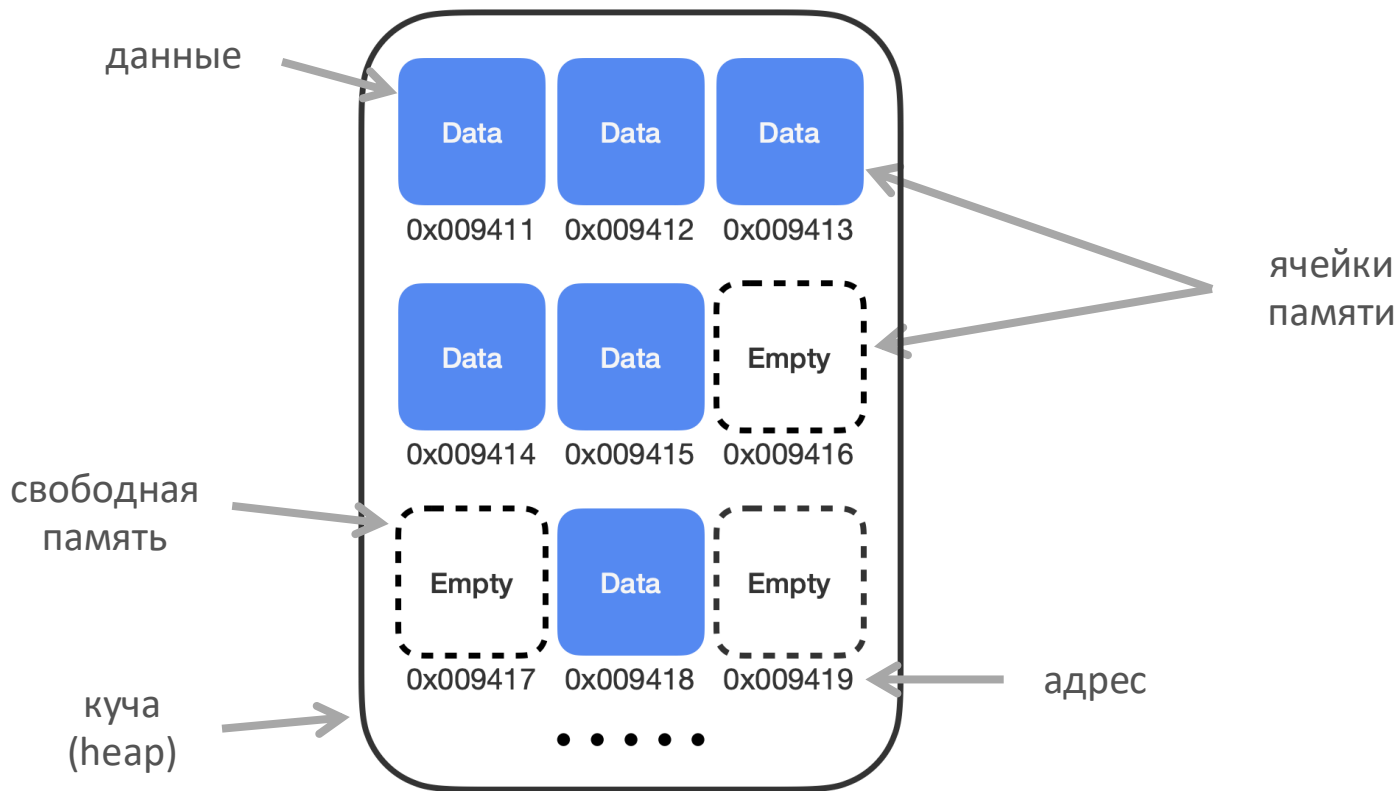
Разделы памяти



Reference type в памяти

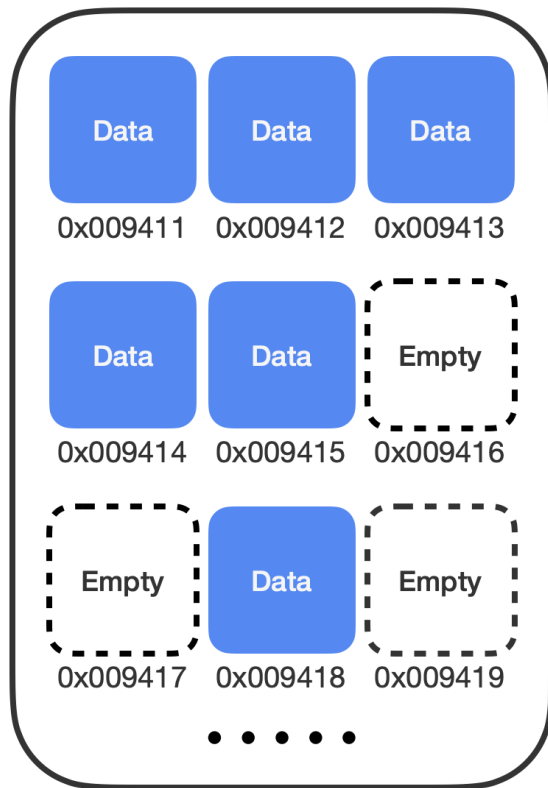


Reference type в памяти



Reference type в памяти

```
class Person {  
    let name: String  
    init(name: String) {  
        self.name = name  
    }  
}
```

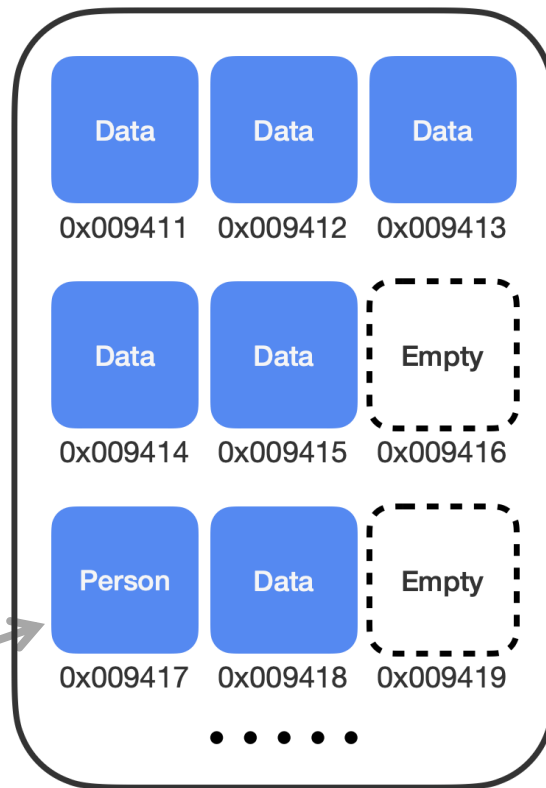


Reference type в памяти

```
class Person {  
    let name: String  
    init(name: String) {  
        self.name = name  
    }  
}
```

```
var person: Person? = Person(name: "Lily")
```

занимаем
память



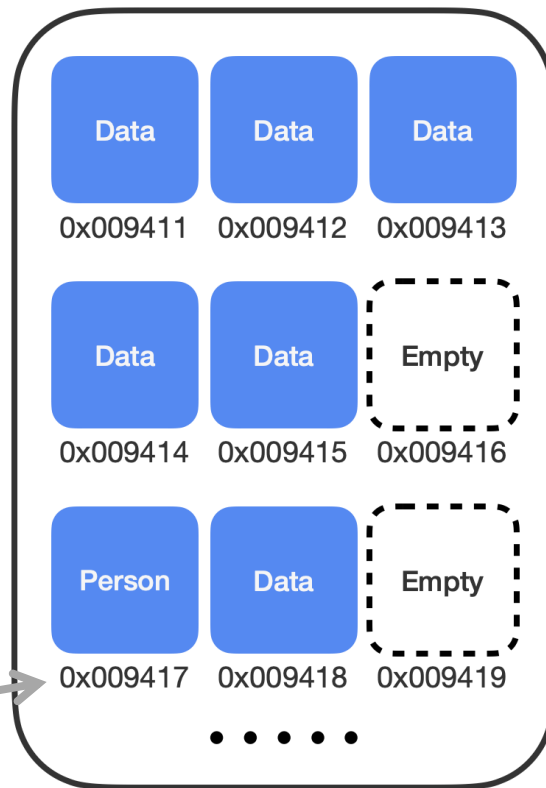
Reference type в памяти

```
class Person {  
    let name: String  
    init(name: String) {  
        self.name = name  
    }  
}
```

```
var person: Person? = Person(name: "Lily")  
// person = 0x009417
```

храним
адрес

адрес в памяти и
переменной совпадает



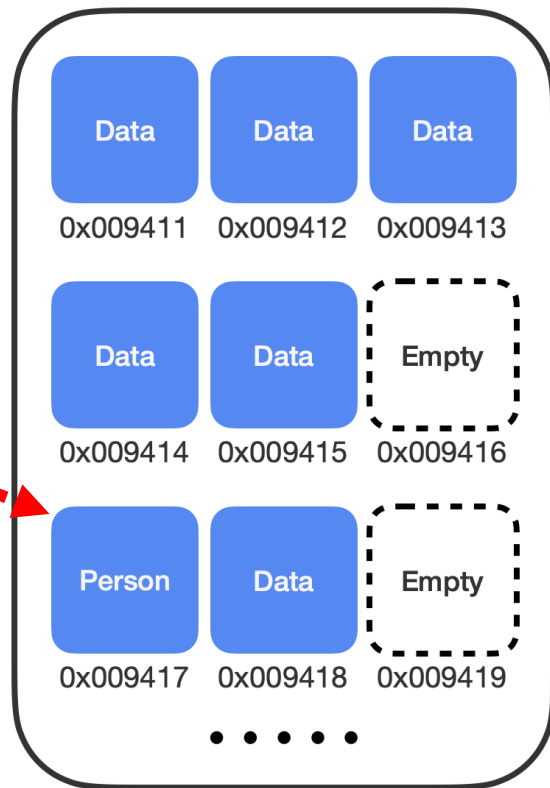
Reference type в памяти

```
class Person {  
    let name: String  
    init(name: String) {  
        self.name = name  
    }  
}
```

```
var person: Person? = Person(name: "Lily")  
// person = 0x009417
```

↑
храним
адрес

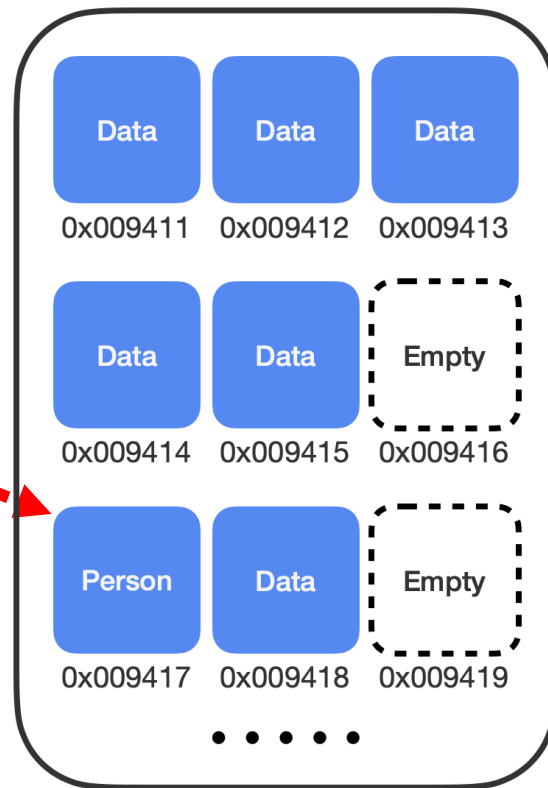
↑
ссылка на ячейку в
памяти



Reference type в памяти

```
class Person {  
    let name: String  
    init(name: String) {  
        self.name = name  
    }  
}
```

```
var person: Person? = Person(name: "Lily")  
// do somethink  
// ....  
//
```

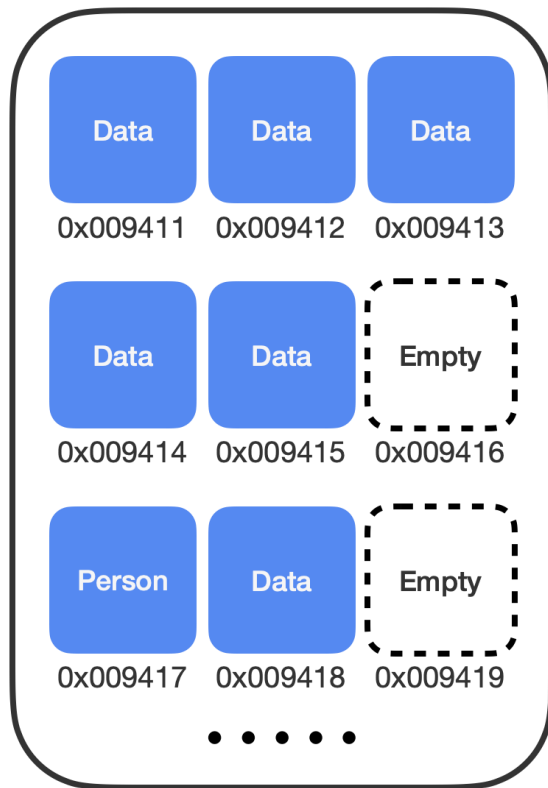


Reference type в памяти

```
class Person {  
    let name: String  
    init(name: String) {  
        self.name = name  
    }  
}  
  
var person: Person? = Person(name: "Lily")  
// do somethink  
// ....  
//  
person = nil
```



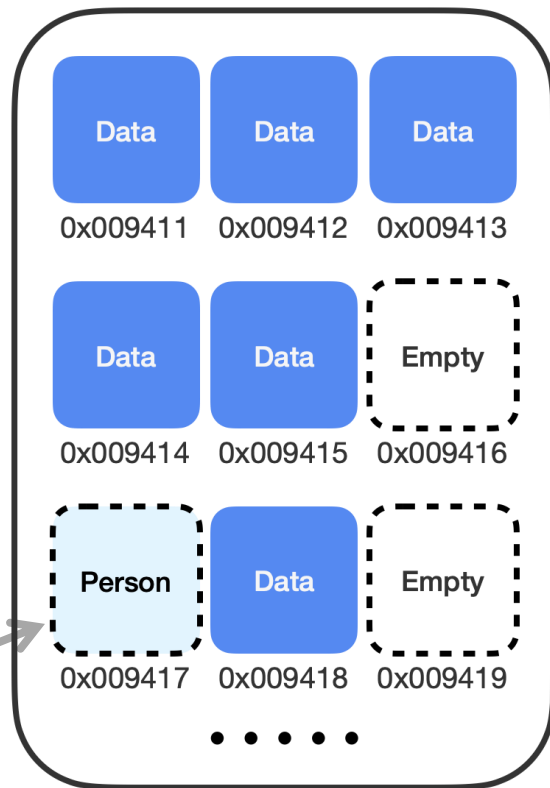
убираем ссылку на
объект



Reference type в памяти

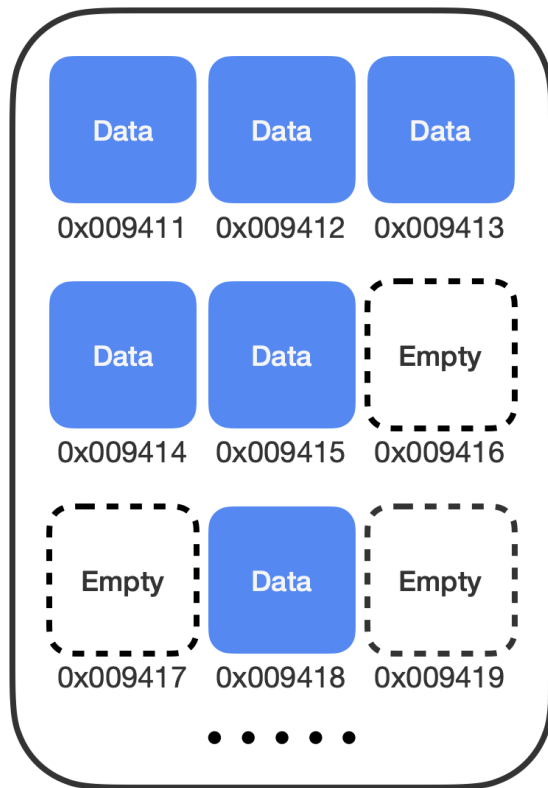
```
class Person {  
    let name: String  
    init(name: String) {  
        self.name = name  
    }  
}  
  
var person: Person? = Person(name: "Lily")  
// do somethink  
// ....  
//  
person = nil
```

ОЧИСТКА



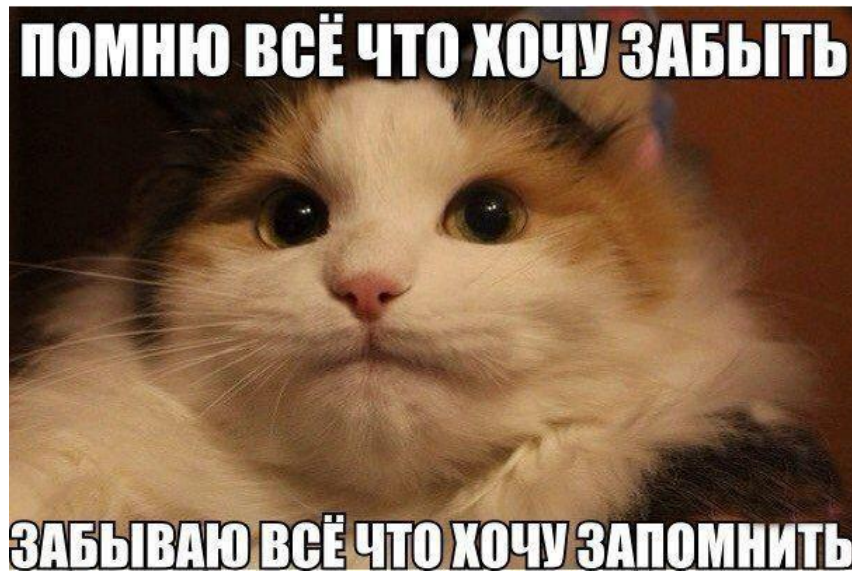
Reference type в памяти

```
class Person {  
    let name: String  
    init(name: String) {  
        self.name = name  
    }  
}  
  
var person: Person? = Person(name: "Lily")  
// do somethink  
// ....  
//  
person = nil
```



MRC — Manual Reference Count

- Нужно вручную следить за количеством ссылок на каждый объект
- Если останется не отпущенная ссылка на объект, то произойдёт утечка памяти



Будет ли утечка памяти?

```
@interface ViewController ()

@property (retain, nonatomic) UITextField *someProperty;

@end

self.someProperty = [[UITextField alloc] init];
[_someProperty retain];

- (void)dealloc
{
    [_someProperty release];
    [super dealloc];
}
```


Будет ли утечка памяти?

```
@interface ViewController ()

@property (retain, nonatomic) UITextField *someProperty;

@end

self.someProperty = [[UITextField alloc] init];

[_someProperty retain];

- (void)dealloc
{
    [_someProperty release];
    [_someProperty release];
    [super dealloc];
}
```

Теперь точно всё?!

```
@interface ViewController ()

@property (retain, nonatomic) UITextField *someProperty;

@end

self.someProperty = [[UITextField alloc] init];

[_someProperty retain];

- (void)dealloc
{
    [_someProperty release];
    [_someProperty release];
    [_someProperty release];
    [super dealloc];
}
```

ARC (Automatic Reference Count)

- Начинает работу на этапе компиляции и проставляет retain и release объектам
- считает количество ссылок на объект в ран-тайме
- деаллоцирует объект, когда он больше не нужен (количество ссылок на объект = 0)



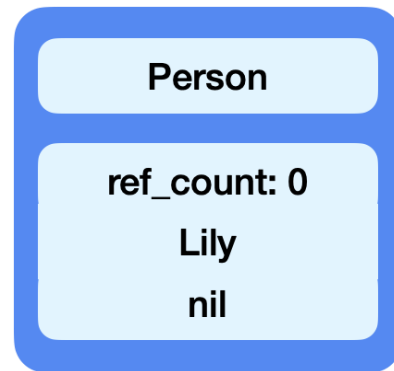
Пример работы ARC

```
class Person {  
    let name: String  
    var account: String?  
  
    init(name: String) {  
        self.name = name  
    }  
}
```

```
func test() {  
    let person1 = Person(name: "Lily")  
  
    let person2 = person1  
  
    person2.account = "Pukula"  
  
    print("Done")  
}
```

Пример работы ARC

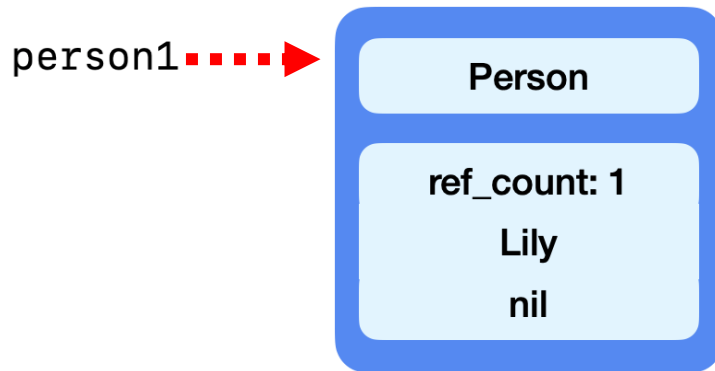
```
func test() {  
    let person1 = Person(name: "Lily")  
      
    let person2 = person1  
      
    person2.account = "Pukula"  
      
    print("Done")  
}
```



ref_count — количество
ссылок на объект в памяти

Пример работы ARC

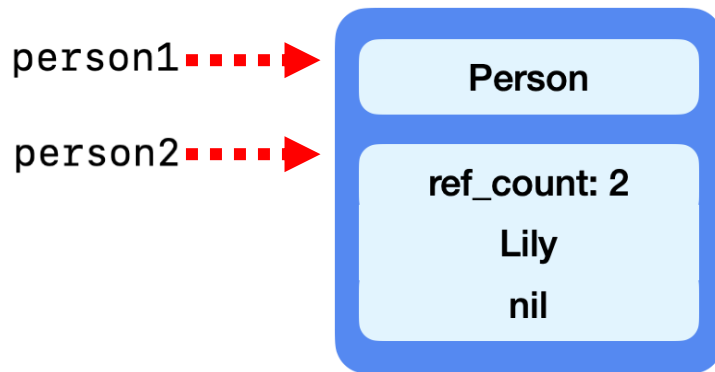
```
func test() {  
    let person1 = Person(name: "Lily")  
      
    let person2 = person1  
      
    person2.account = "Pukula"  
      
    print("Done")  
}
```



ref_count — количество
ссылок на объект в памяти

Пример работы ARC

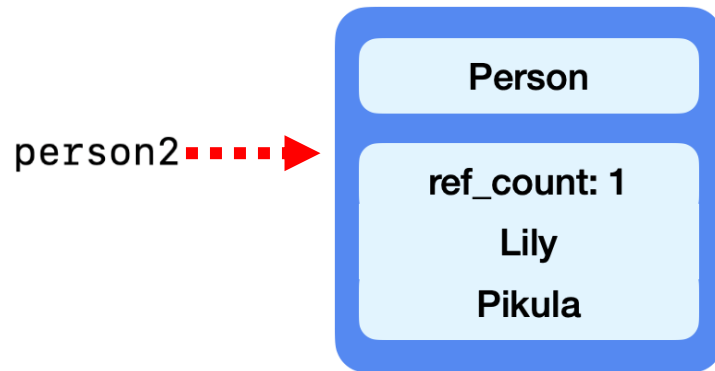
```
func test() {  
    let person1 = Person(name: "Lily")  
    [REDACTED]  
    let person2 = person1  
    [REDACTED]  
    person2.account = "Pukula"  
    [REDACTED]  
    print("Done")  
}
```



`ref_count` — количество
ссылок на объект в памяти

Пример работы ARC

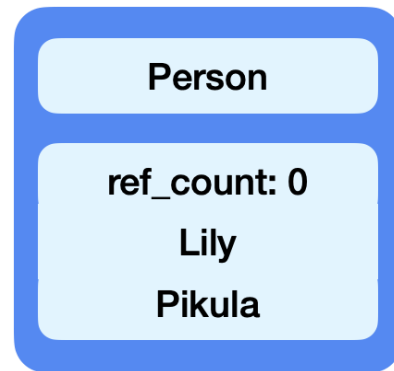
```
func test() {  
    let person1 = Person(name: "Lily")  
      
    let person2 = person1  
      
    person2.account = "Pukula"  
      
    print("Done")  
}
```



ref_count — количество
ссылок на объект в памяти

Пример работы ARC

```
func test() {  
    let person1 = Person(name: "Lily")  
    ██████████  
    let person2 = person1  
    ██████████  
    person2.account = "Pukula"  
    ██████████  
    print("Done")  
}
```



ref_count — количество
ссылок на объект в памяти

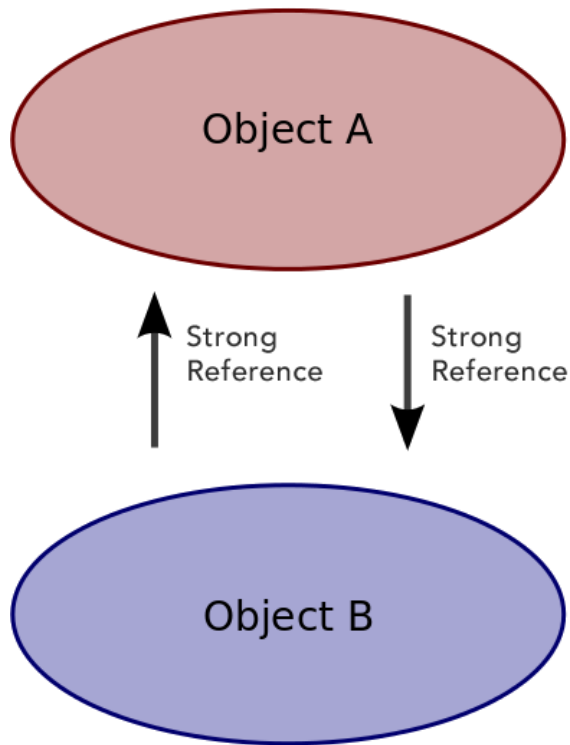
Пример работы ARC

```
func test() {  
    let person1 = Person(name: "Lily")  
      
    let person2 = person1  
      
    person2.account = "Pukula"  
      
    print("Done")  
}
```

Объект очищен из
памяти, потому что
ref_count = 0

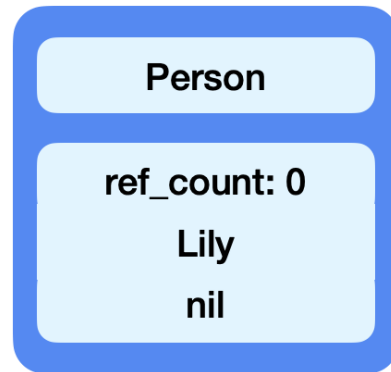
ARC: Retain Cycle

```
class Person {  
    let name: String  
    var account: Account?  
  
    init(name: String) {  
        self.name = name  
    }  
  
    func printAccount() {  
        guard let account else { return }  
  
        print("Nickname:", account.nickname)  
    }  
}
```



ARC: Retain Cycle

```
class Person {  
    let name: String  
    var account: Account?  
  
    func printAccount() { /*...*/ }  
}  
  
class Account {  
    var person: Person  
    var nickname: String  
}  
  
func test() {  
    let person = Person(name: "Lily")  
    let account = Account(  
        person: person,  
        nickname: "Pikula"  
    )  
    person.account = account  
    person.printAccount()  
}
```



ARC: Retain Cycle

```
class Person {  
    let name: String  
    var account: Account?  
  
    func printAccount() { /*...*/ }  
}
```

```
class Account {  
    var person: Person  
    var nickname: String  
}
```

```
func test() {  
    let person = Person(name: "Lily")  
    let account = Account(  
        person: person,  
        nickname: "Pikula"  
    )  
    person.account = account  
    person.printAccount()  
}
```

person 

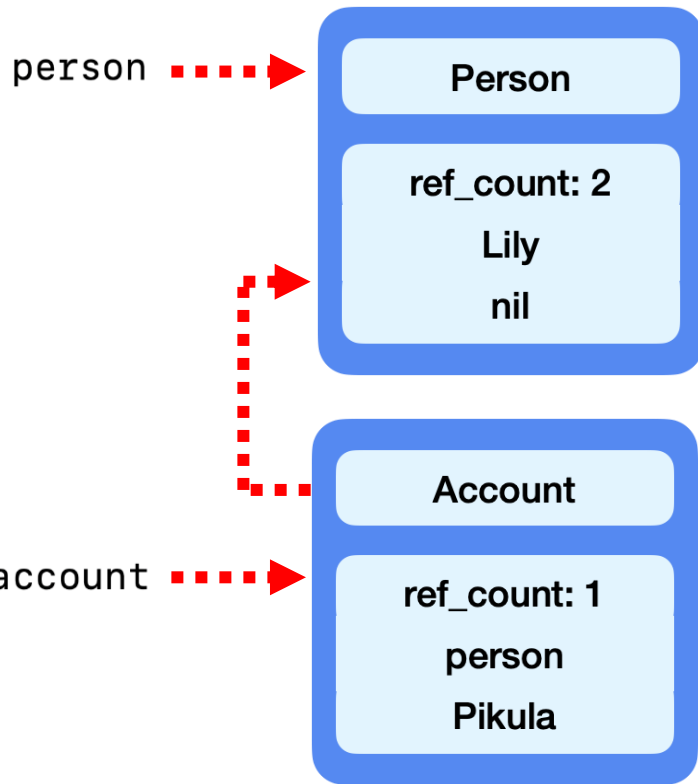


ARC: Retain Cycle

```
class Person {  
    let name: String  
    var account: Account?  
  
    func printAccount() { /*...*/ }  
}
```

```
class Account {  
    var person: Person  
    var nickname: String  
}
```

```
func test() {  
    let person = Person(name: "Lily")  
    let account = Account(  
        person: person,  
        nickname: "Pikula"  
    )  
    person.account = account  
    person.printAccount()  
}
```

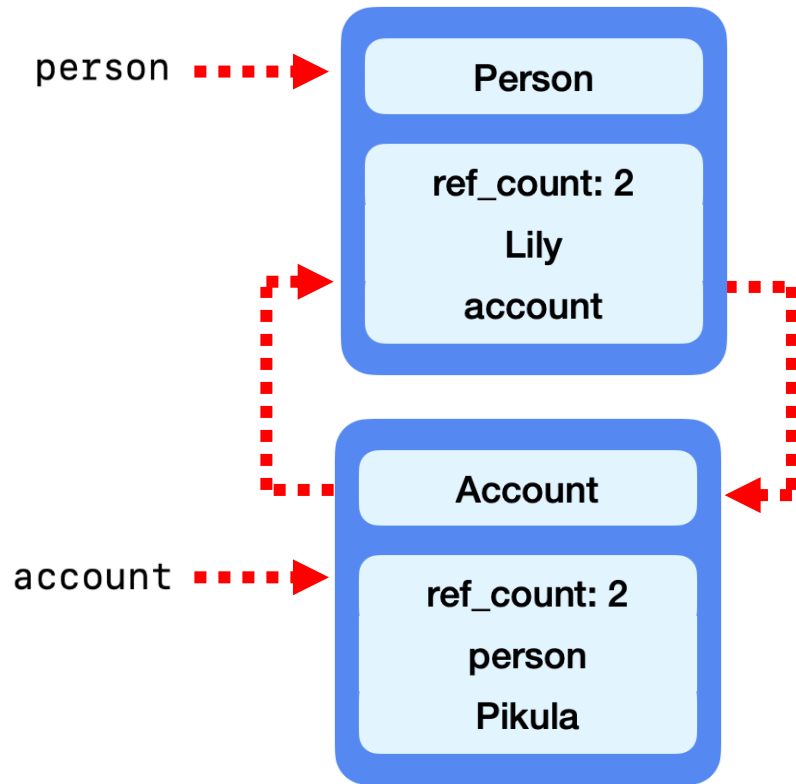


ARC: Retain Cycle

```
class Person {  
    let name: String  
    var account: Account?  
  
    func printAccount() { /*...*/ }  
}
```

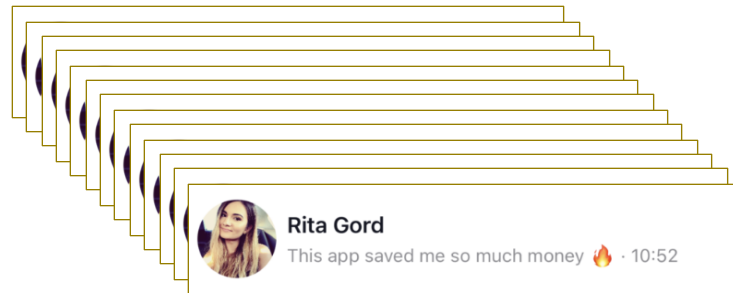
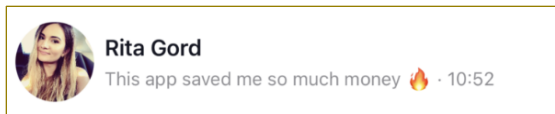
```
class Account {  
    var person: Person  
    var nickname: String  
}
```

```
func test() {  
    let person = Person(name: "Lily")  
    let account = Account(  
        person: person,  
        nickname: "Pikula"  
    )  
    person.account = account  
    person.printAccount()  
}
```



ARC: Retain Cycle — последствия

- **Утечка памяти:** Память, выделенная под неосвобождаемые объекты, не возвращается обратно в память, что может привести к исчерпанию ресурсов



- **Падение производительности:** Процессор тратит время на обработку неиспользуемых данных
- **Нестабильность приложения:** Приложение может начать работать некорректно или даже зависнуть, если утечка памяти становится значительной

ARC: Слабые и бесхозные ссылки

По умолчанию все ссылки strong, поэтому для решения проблемы есть другой тип ссылок.

weak & **unowned** — решают проблему Retain Cycle, позволяя создавать ссылки на объект, которые **не увеличивают** его счетчик ссылок

иначе говоря **не триггерят** изменение значения **ref_count**

ARC: Retain Cycle — closure

```
class MyClass {  
    let name: String  
    var myClosure: (() -> Void)?  
  
    init(name: String) {  
        self.name = name  
    }  
}
```

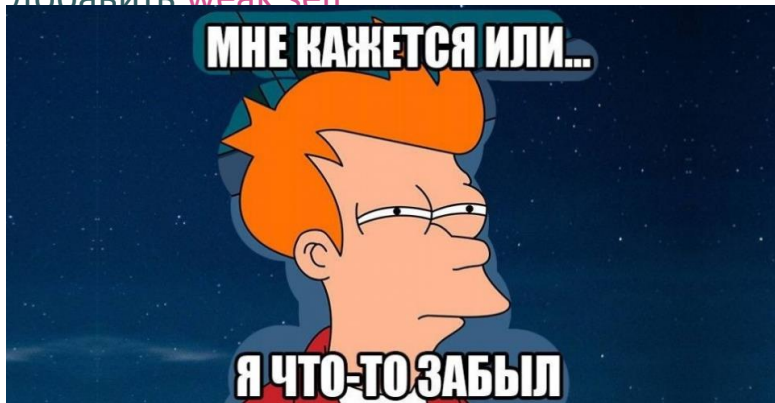
```
func createRetainCycle() {  
    myClosure = { [weak self] in  
        print(self?.name ?? "")  
    }  
}
```

```
deinit {  
    print("Deinitialized")  
}
```

```
var myObject: MyClass? = MyClass(  
    name: "Example"  
)  
myObject?.createRetainCycle()  
myObject = nil
```

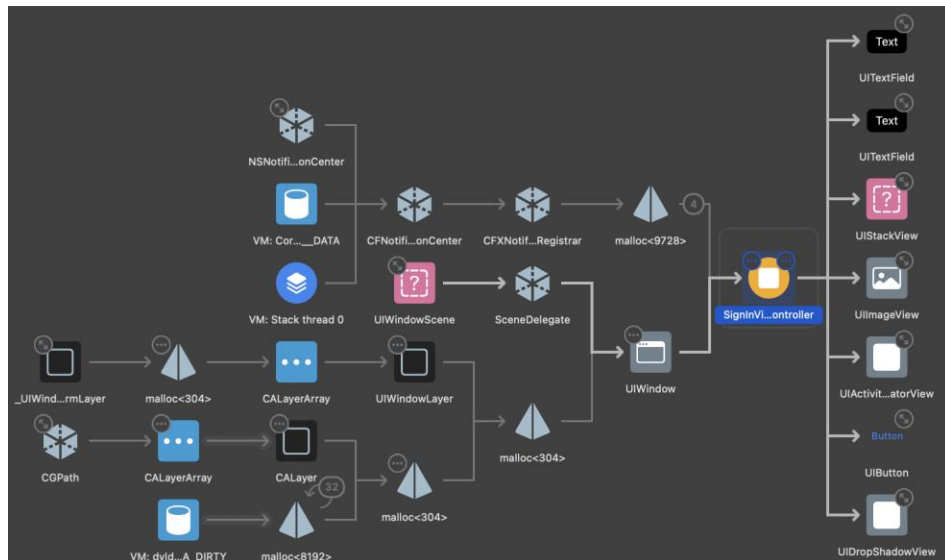
Для решения
проблемы надо
добавить **weak self**

deinit теперь
вызывается



ARC: Retain Cycle — как обнаружить?

- Ждать когда приложение начнет тормозить
- Проверить вызов **deinit**
- Использовать Debug Memory Graph



Итоги

В этом разделе мы узнали:

- Как работает память у reference type данных
- Что такое MRC, детально рассмотрели работу ARC
- Что такое Retain Cycle, как возникает и как с ним бороться
- Что такое **weak** и **unowned**
- Как можно находить Retain Cycle в своих проектах

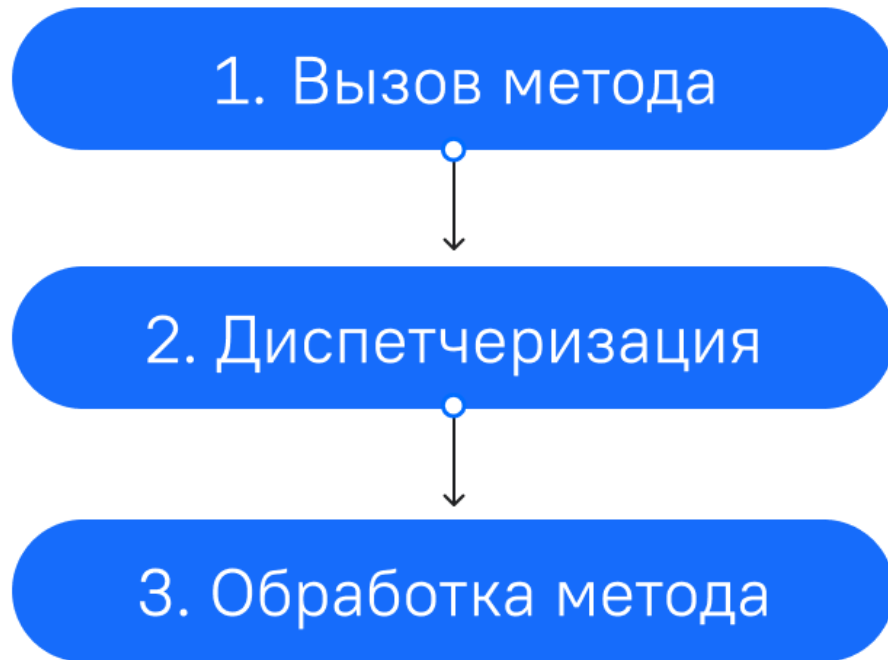
Вопросы?



Диспетчеризация

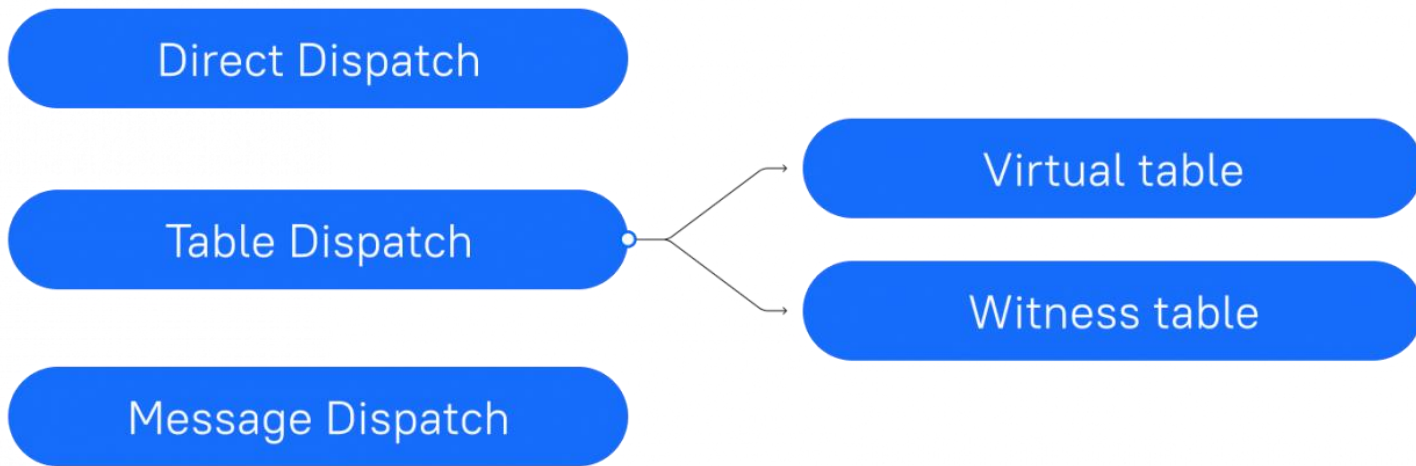


Диспетчеризация



Цель диспетчеризации состоит в том, чтобы программа сообщила центральному процессору, где в памяти он может найти исполняемый код для вызова конкретного метода

Типы диспетчеризации



Цель диспетчеризации состоит в том, чтобы программа сообщила центральному процессору, где в памяти он может найти исполняемый код для вызова конкретного метода

Direct dispatch

Плюсы:

Самая быстрая из типов
диспетчеризаций

Минусы:

Принципы ООП теряются
(наследование и всё,
что из него исходит)

Final Class

```
// MARK: Example 1 – Final Class
final class ClassExample1 {
    // MARK: Direct Dispatch
    func doSomething() {
        print("Example 1 – Final Class")
    }
}
```

Protocol Extension

```
// MARK: Example 2 – Protocol Extension
extension SomeProtocol {
    // MARK: Direct Dispatch
    func doSomething() {
        print("Example 2 – Protocol Extension")
    }
}

class ClassExample2: SomeProtocol {}

let classExample2 = ClassExample2()
classExample2.doSomething()
```

Table dispatch (Virtual table)

Плюсы:

Принципы ООП
(наследование и всё,
что из него исходит)

Минусы:

Скорость
существенно ниже,
чем у Direct Dispatch

```
// MARK: Example 5 - Virtual Table
class ClassExample5 {
    func doSomething() {
        print("Example 5 - Virtual Table")
    }
}

class SubclassExample5: ClassExample5 {
    override func doSomething() {
        print("Override for subclass")
    }

    func doSomething2() {
        print("Method of subclass")
    }
}
```

ClassExample5

ClassExample5	0xxBC0
doSomething()	0xxC12

SubclassExample5

SubclassExample5	0xxBC0
doSomething	0xxD12
doSomething2	0xxB22

Table dispatch (Witness table)

Плюсы:

Полиморфизм

Минусы:

Скорость
существенно ниже,
чем у Direct Dispatch
и равна скорости
Virtual Table

Отсутствует
наследование

```
// MARK: Example 6 - Witness Table
```

```
protocol ProtocolExample6 {
```

```
    func doSomething()
```

```
}
```

```
class ClassExample6: ProtocolExample6 {
```

```
    func doSomething() {
```

```
        print("Example 6 - Witness Table")
```

```
    }
```

```
}
```

```
class AnotherClassExample6: ProtocolExample6 {
```

```
    func doSomething() {
```

```
        print("Hello World")
```

```
    }
```

```
}
```

ClassExample6

ProtocolExample6	0xCD0
doSomething	0xB10

AnotherClassExample6

ProtocolExample6	0xCD0
doSomething	0xB14

Message dispatch

Плюсы:

Method Swizzling

Наивысшая
динамичность

Минусы:

Самая низкая скорость

```
class ClassExample7: NSObject {  
    @objc dynamic func doSomething() {  
        print("Example 7 – Message Dispatch")  
    }  
}
```

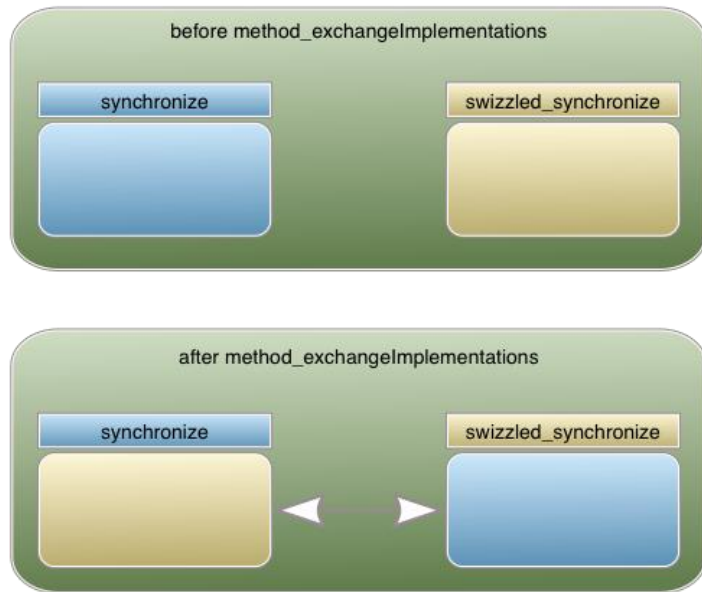
Method swizzling

Swizzling — это процесс изменения реализации существующего метода

```
let originalSelector = #selector(originalMethod)
let customSelector = #selector(customMethod)
```

```
guard let original = class_getInstanceMethod(Self.self, originalSelector),
      let custom = class_getInstanceMethod(Self.self, customSelector) else { return }
```

```
method_exchangeImplementations(original, custom)
```



Method swizzling

Минусов у данного подхода больше, поэтому



Итоги

В этом разделе мы узнали:

- Что такое диспетчеризация и зачем она нужна
- Какой тип диспетчеризации самый быстрый
- Когда использовать `message dispatch`
- `Method swizzling` (оно вам не надо)

Вопросы?





Спасибо 🎉

